

1. Objective

Implement KNN from scratch in Python and test how different values of K affect classification accuracy on a diabetes dataset.

2. Dataset

Pima Indians Diabetes Dataset — 768 samples, 8 input features, binary output (diabetic / not diabetic).

Features: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age.

Split: 70% training (537 samples) | 30% testing (231 samples)

3. Implementation Steps

- Normalize features using min-max normalization (range 0 to 1).
- Randomly shuffle and split data into train/test sets (seed = 112320065).
- For each test point, compute Euclidean distance to all training points.
- Select K nearest neighbors and assign the majority class label.
- Repeat for K = 1, 3, 5, 7 and compare accuracy.

4. Results

The table below shows accuracy for each K value:

K Value	Correct	Total	Accuracy
K = 1	161	231	69.70%
K = 3	171	231	74.03%
K = 5	171	231	74.03%
K = 7	170	231	73.59%

5. K vs Accuracy Plot

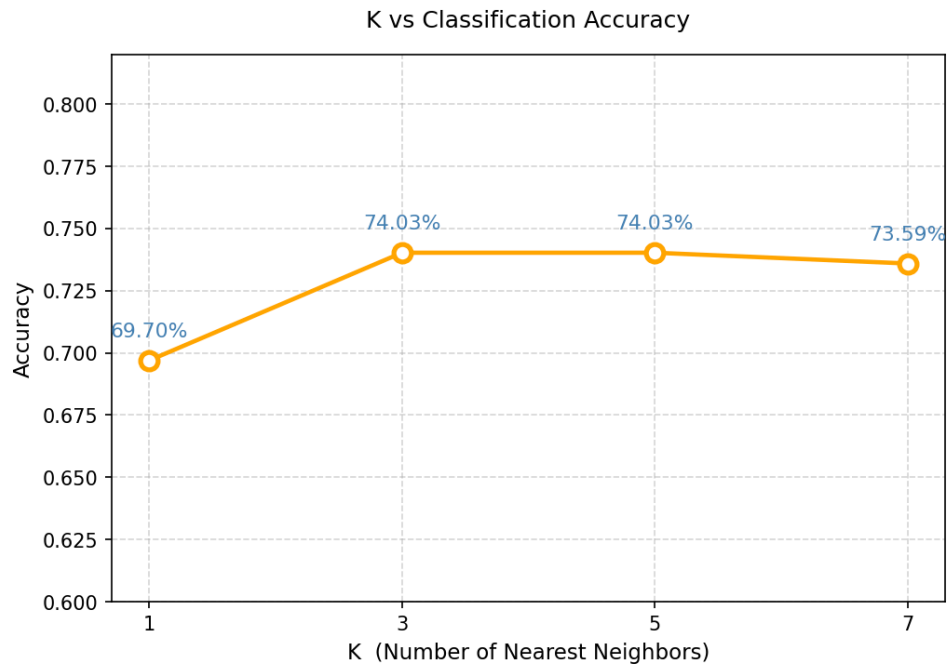


Figure 1: Accuracy increases from $K=1$ to $K=3$, then stays stable

6. Prediction Breakdown for Each K

Each chart shows True Positive, True Negative, False Positive, and False Negative counts.

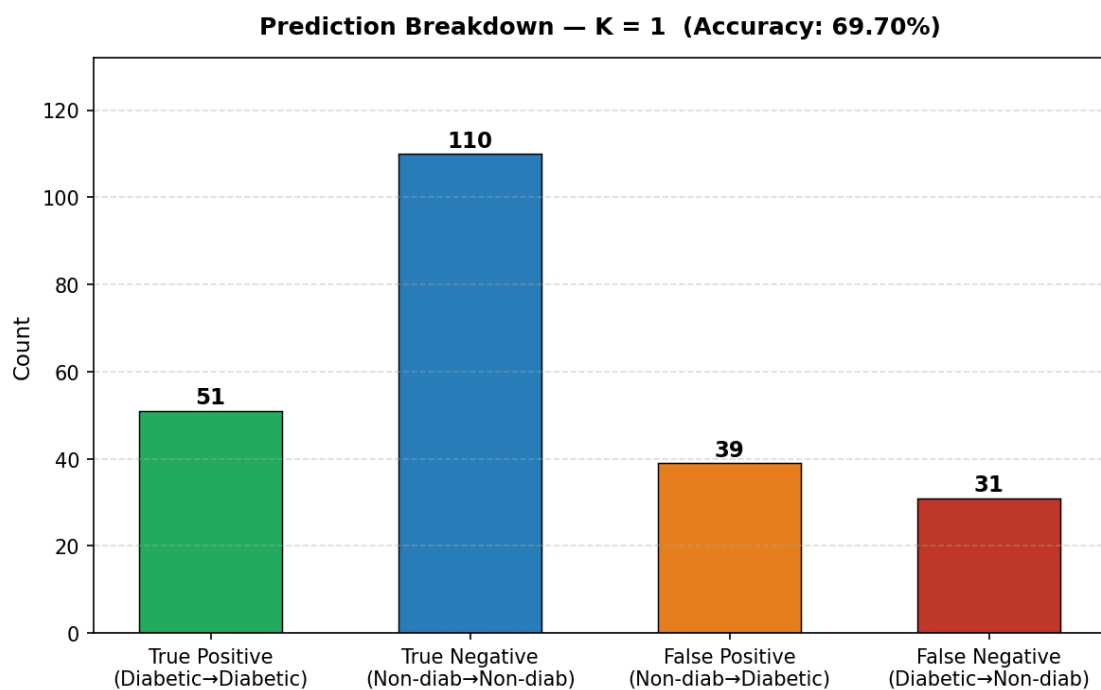


Figure 2: $K = 1$ (Accuracy: 69.70%)

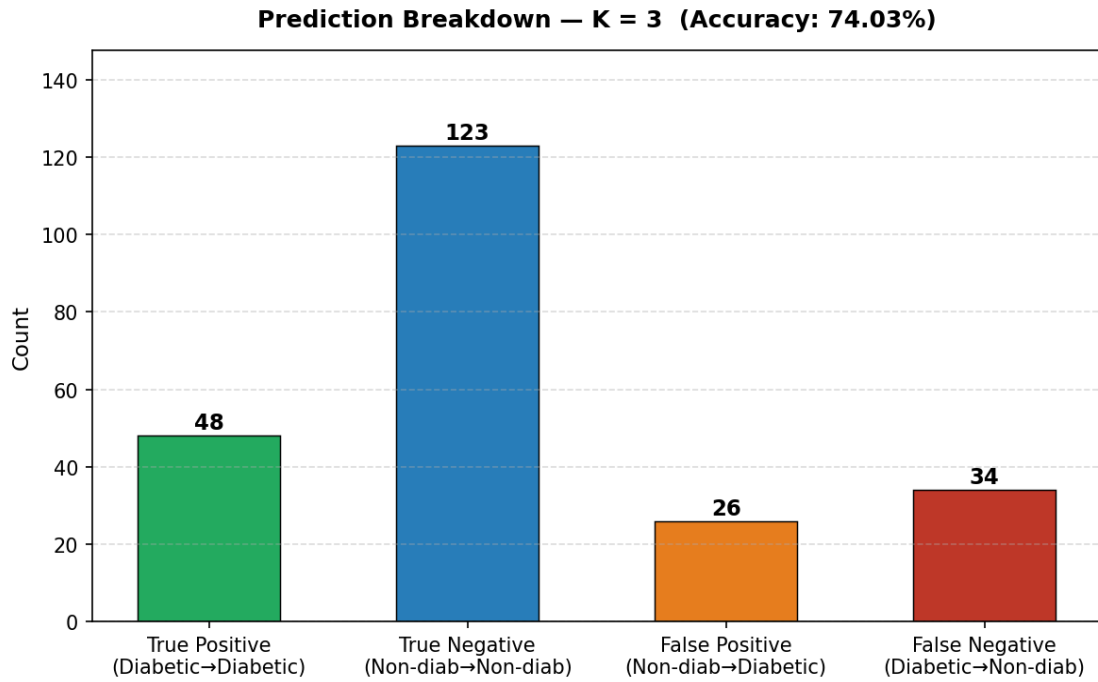


Figure 3: K = 3 (Accuracy: 74.03%)

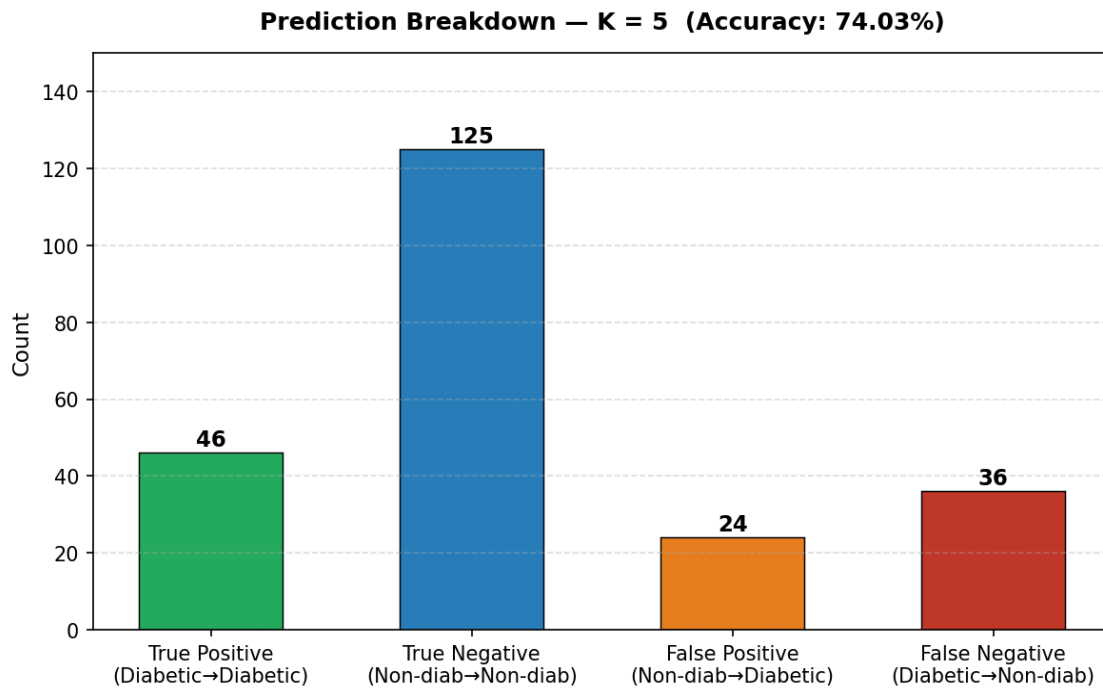


Figure 4: K = 5 (Accuracy: 74.03%)

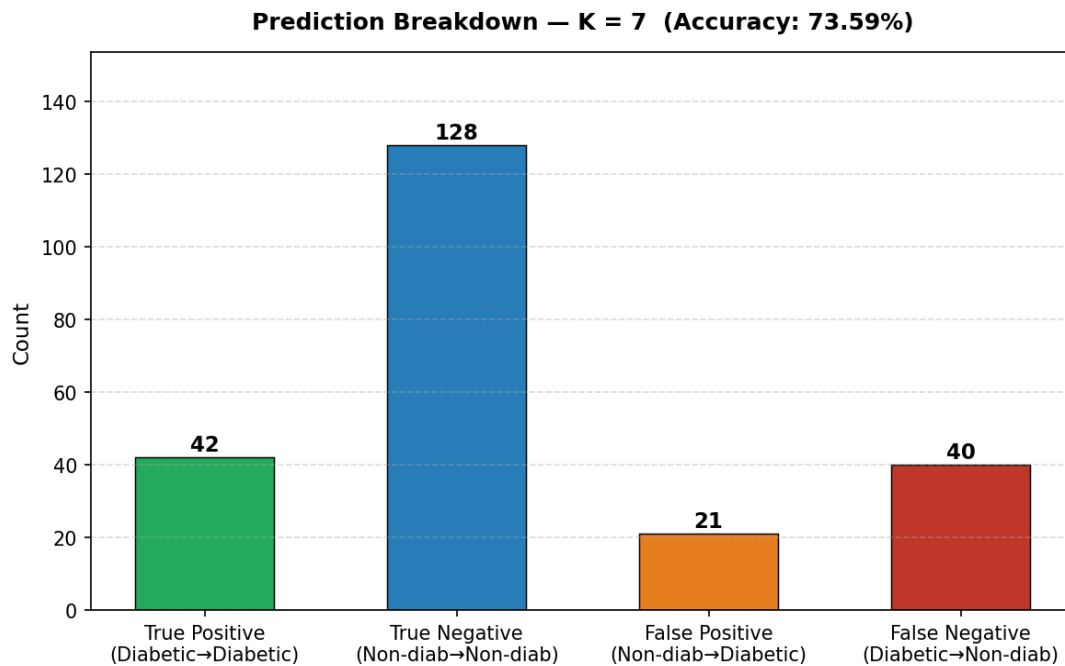


Figure 5: K = 7 (Accuracy: 73.59%)

7. Conclusion

- K = 1 gave the lowest accuracy (69.70%) — too sensitive to noisy data.
- K = 3 and K = 5 both gave the best accuracy of 74.03%.
- K = 7 dropped slightly to 73.59% — too many neighbors hurts precision.
- Best choice for this dataset: K = 3.